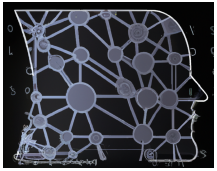
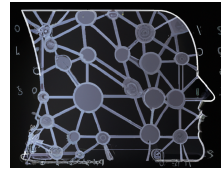
Applied Machine Learning

Applied Hyperparameter Tuning



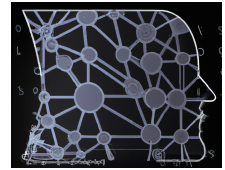
Learning goals

- The Hyperparameter Optimization Problem
- Grid Search, Random Search and Bayesian optimization
- The Untouched Test Set Principle

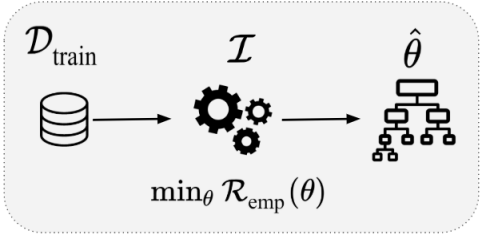


Hyperparameter Tuning

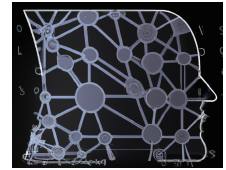
MOTIVATING EXAMPLE



- Given a data set, we want to train a classification tree.
- We feel that a maximum tree depth of 4 has worked out well for us previously, so we decide to set this hyperparameter to 4.
- The learner ("inducer") $\mathcal{I}$ takes the input data, internally performs **empirical risk minimization**, and returns a fitted tree model $\hat{f}(\mathbf{x}) = f(\mathbf{x}, \hat{\theta})$ of at most depth $\lambda = 4$ that minimizes empirical risk.

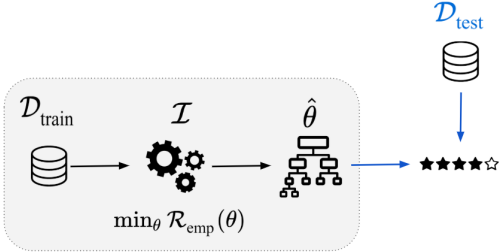


MOTIVATING EXAMPLE / 2

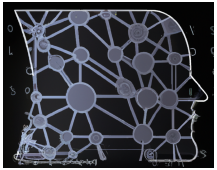


- We are **actually** interested in the **generalization performance** $\mathrm{GE}\left(\hat{f}\right)$ of the estimated model on new, previously unseen data.
- We estimate the generalization performance by evaluating the model $\hat{f} = \mathcal{I}(\mathcal{D}_{\text{train}}, \boldsymbol{\lambda})$ on a test set $\mathcal{D}_{\text{test}}$:

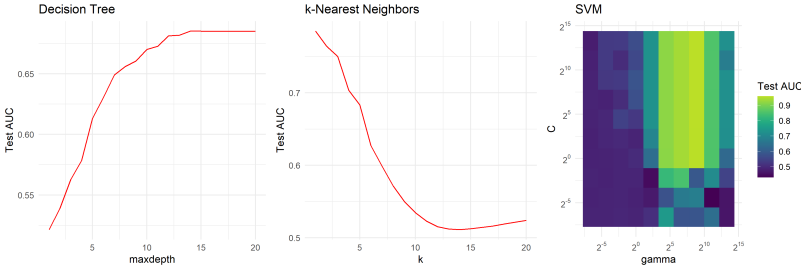
$$\widehat{\mathrm{GE}}_{\mathcal{D}_{\text{train}},\mathcal{D}_{\text{test}}}(\mathcal{I}, \boldsymbol{\lambda}, n_{\text{train}}, \rho) = \rho\left(\mathbf{y}_{\mathcal{D}_{\text{test}}}, \boldsymbol{F}_{\mathcal{D}_{\text{test}},\hat{f}}\right)$$



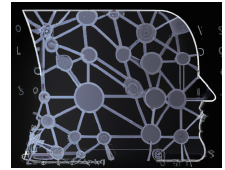
MOTIVATING EXAMPLE / 3



- But many ML algorithms are sensitive w.r.t. a good setting of their hyperparameters, and generalization performance might be bad if we have chosen a suboptimal configuration.
- Consider a simulation example of 3 ML algorithms below, where we use the dataset *mlbench.spiral* and 10,000 testing points. As can be seen, variating hyperparameters can lead to big difference in model's generalization performance.



MOTIVATING EXAMPLE / 4

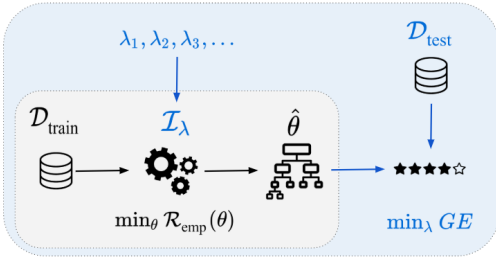


For our example this could mean:

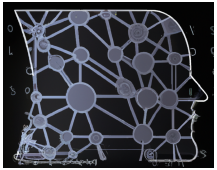
- Data too complex to be modeled by a tree of depth 4
- Data much simpler than we thought, a tree of depth 4 overfits

$\Longrightarrow$ Algorithmically try out different values for the tree depth. For each maximum depth λ, we have to train the model **to completion** and evaluate its performance on the test set.

- We choose the tree depth λ that is **optimal** w.r.t. the generalization error of the model.



MODEL PARAMETERS VS. HYPERPARAMETERS



It is critical to understand the difference between model parameters and hyperparameters.

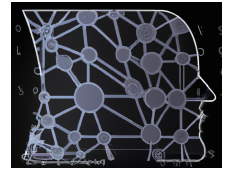
Model parameters $\boldsymbol{\theta}$ are optimized during training. They are an **output** of the training.

Examples:

- The splits and terminal node constants of a tree learner
- Coefficients $\boldsymbol{\theta}$ of a linear model $f(\mathbf{x}) = \boldsymbol{\theta}^\top \mathbf{x}$

MODEL PARAMETERS VS. HYPERPARAMETERS

/ **2**



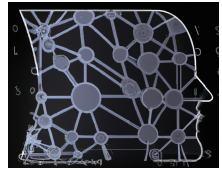
In contrast, **hyperparameters** (HPs) λ are not optimized during training. They must be specified in advance, are an **input** of the training. Hyperparameters often control the complexity of a model, i.e., how flexible the model is. They can in principle influence any structural property of a model or computational part of the training process.

The process of finding the best hyperparameters is called **tuning**.

Examples:

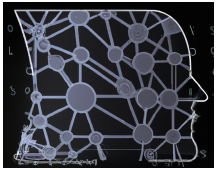
- Maximum depth of a tree
- k and which distance measure to use for k-NN
- Number and maximal order of interactions to be included in a linear regression model

MODEL PARAMETERS VS. HYPERPARAMETERS / 3



- Number of optimization steps if the empirical risk minimization is done via gradient descent

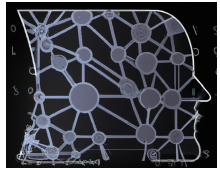
TYPES OF HYPERPARAMETERS



We summarize all hyperparameters we want to tune in a vector $\boldsymbol{\lambda} \in \boldsymbol{\Lambda}$ of (possibly) mixed type. HPs can have different types:

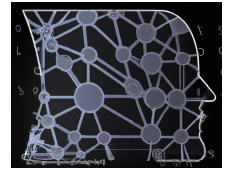
- Real-valued parameters, e.g.:
  - Minimal error improvement in a tree to accept a split
  - Bandwidths of the kernel density estimates for Naive Bayes
- Integer parameters, e.g.:
  - Neighborhood size k for k-NN
  - *mtry* in a random forest
- Categorical parameters, e.g.:
  - Which split criterion for classification trees?
  - Which distance measure for k-NN?

Hyperparameters are often **hierarchically dependent** on each other, e.g., *if* we use a kernel-density estimate for Naive Bayes, what is its width?

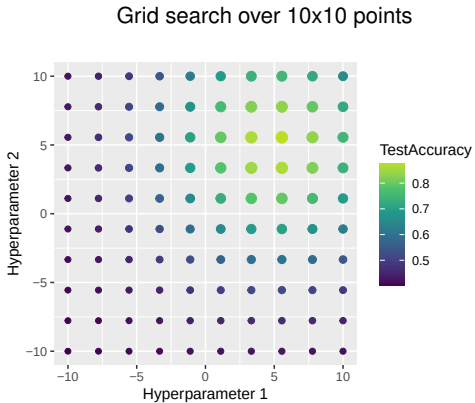


Hyperparameter Tuning Algorithms

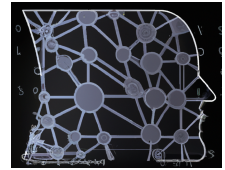
GRID SEARCH



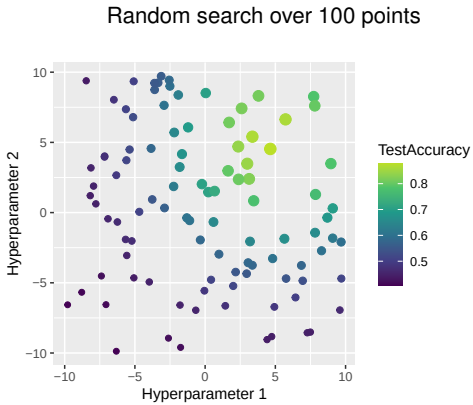
- Simple technique which is still quite popular, tries all HP combinations on a multi-dimensional discretized grid
- For each hyperparameter a finite set of candidates is predefined
- Then, we simply search all possible combinations in arbitrary order



RANDOM SEARCH

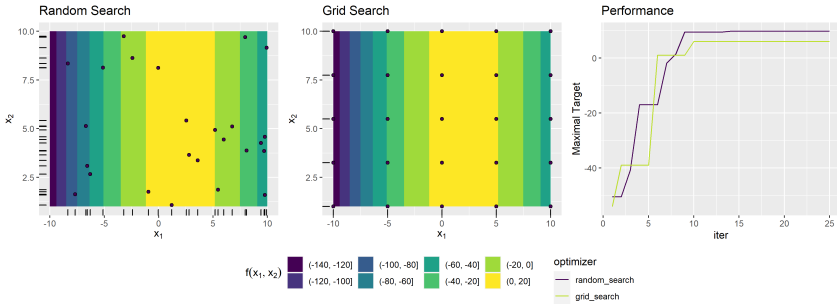


- Small variation of grid search
- Uniformly sample from the region-of-interest



RANDOM SEARCH VS. GRID SEARCH

We consider a maximization problem on the function $f(x_1, x_2) = g(x_1) + h(x_2) \approx g(x_1)$, i.e. in order to maximize the target, x_1 should be the parameter to focus on.

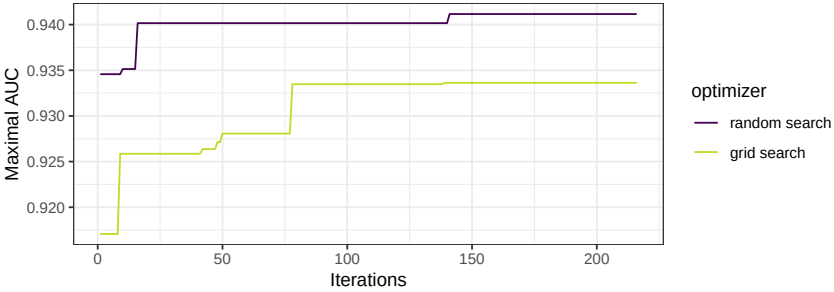


$\Rightarrow$ In this setting, random search is more superior as we get a better coverage for the parameter x_1 in comparison with grid search, where we only discover 5 distinct values for x_1.

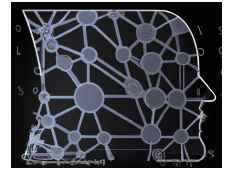
TUNING EXAMPLE

Tuning random forest with grid search/random search and 5CV on the `sonar` data set for AUC:

Hyperparameter	Type	Min	Max
`num.trees`	integer	3	500
`mtry`	integer	5	50
`min.node.size`	integer	10	100

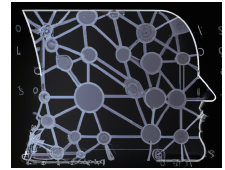


SUMMARY

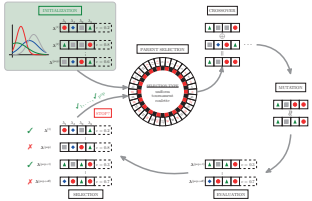


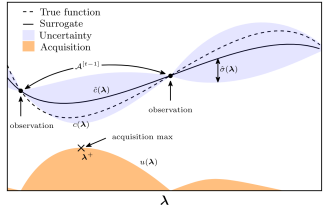
Property	Grid Search	Random Search
Easy to implement	✓	✓
All parameter types possible	✓	✓
Parallelization is trivial		✓
Does not suffer from the curse of dimensionality		✓
Does not require discretization of hyperparameters		✓
Anytime algorithm		✓

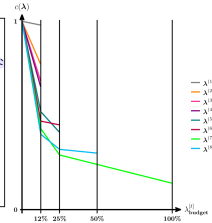
HPO – MANY APPROACHES



- Evolutionary algorithms
- Bayesian / model-based optimization
- Multi-fidelity optimization, e.g. Hyperband





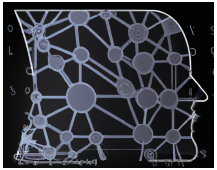


HPO methods can be characterized by:

- how the exploration vs. exploitation trade-off is handled
- how the inference vs. search trade-off is handled

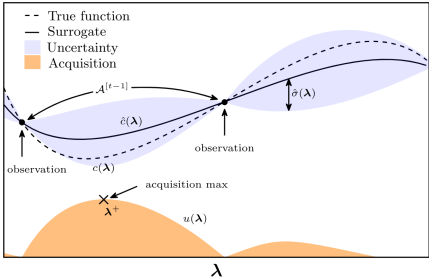
Further aspects: Parallelizability, local vs. global behavior, handling of noisy observations, multifidelity and search space complexity.

BAYESIAN OPTIMIZATION



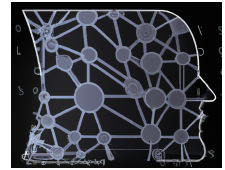
BO sequentially iterates:

1. **Approximate** $\boldsymbol{\lambda} \mapsto c(\boldsymbol{\lambda})$ by (nonlin) regression model $\hat{c}(\boldsymbol{\lambda})$, from evaluated configurations (archive)
2. **Propose candidates** via optimizing an acquisition function that is based on the surrogate $\hat{c}(\boldsymbol{\lambda})$
3. **Evaluate** candidate(s) proposed in 2, then go to 1



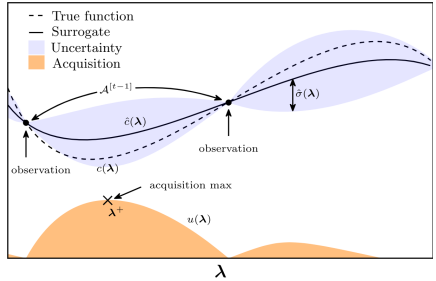
Important trade-off: **Exploration** (evaluate candidates in under-explored areas) vs. **exploitation** (search near promising areas)

BAYESIAN OPTIMIZATION



Surrogate Model:

- Probabilistic modeling of $C(\boldsymbol{\lambda}) \sim (\hat{c}(\boldsymbol{\lambda}), \hat{\sigma}(\boldsymbol{\lambda}))$ with posterior mean $\hat{c}(\boldsymbol{\lambda})$ and uncertainty $\hat{\sigma}(\boldsymbol{\lambda})$.
- Typical choices for numeric spaces are Gaussian Processes; random forests for mixed spaces



Acquisition Function:

- Balance exploration (high $\hat{\sigma}$) vs. exploitation (low $\hat{c}$).
- Lower confidence bound (LCB): $a(\boldsymbol{\lambda}) = \hat{c}(\boldsymbol{\lambda}) - \kappa \cdot \hat{\sigma}(\boldsymbol{\lambda})$
- Expected improvement (EI): $a(\boldsymbol{\lambda}) = \mathbb{E}\left[\max\{c_{\min} - C(\boldsymbol{\lambda}), 0\}\right]$ where ($c_{\min}$ is best cost value from archive)
- Optimizing $a(\boldsymbol{\lambda})$ is still difficult, but cheap(er)

BAYESIAN OPTIMIZATION / 2

Since we use the sequentially updated surrogate model predictions of performance to propose new configurations, we are guided to "interesting" regions of Λ and avoid irrelevant evaluations:

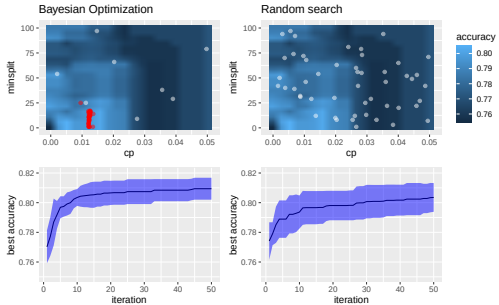
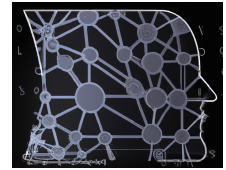


Figure: Tuning complexity and minimal node size for splits for CART on the `titanic` data (10-fold CV maximizing accuracy).
Left panel: BO, 50 configurations; right panel: random search, 50 iterations.
Top panel: one run (initial design of BO is white); bottom panel: mean $\pm$ std of 10 runs.

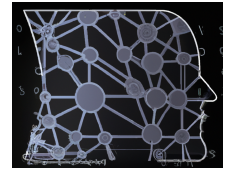
PRACTICAL ASPECTS OF HPO



- Choosing HPO algorithm
  - For few HPS (1-3), grid search can be used
  - BO with GPs for upto 10 numeric HPs
  - BO with RFs handle mixed HP spaces
  - Random search and Hyperband work well as long as the "effective" dimension is low
  - EAs are somewhat in-between BO and RS, can handle very complex spaces, but less sample efficient than BO
  - **Also: use something that's stable and robust! More an aspect of the implementation than the algo!**

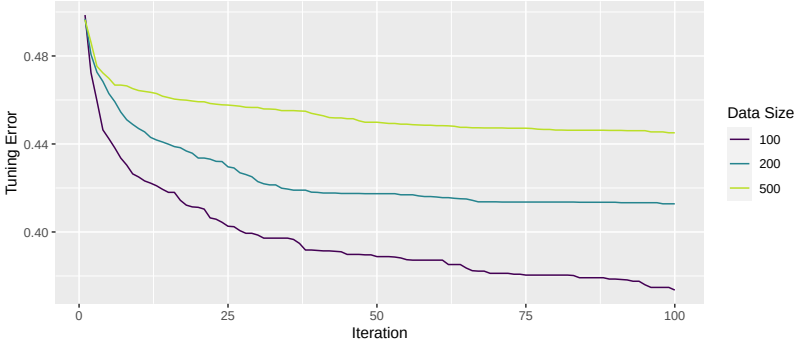
The Untouched Test Set Principle

INSTRUCTIVE AND PROBLEMATIC EXAMPLE



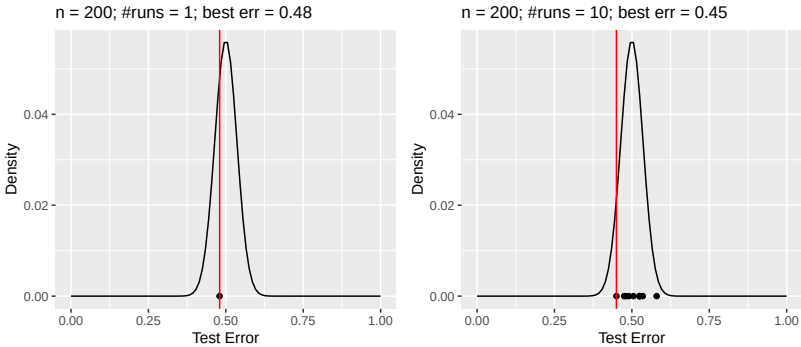
- Assume a binary classification problem with equal class sizes.
- Assume a learner with hyperparameter λ.
- Here, the learner is a (nonsense) feature-independent classifier, where λ has no effect. The learner simply predicts random labels with equal probability.
- Of course, its true generalization error is 50%.
- A cross-validation of the learner (with any fixed λ) will easily show this (given that the partitioned data set for CV is not too small).
- Now let's "tune" it, by trying out 100 different λ values.
- We repeat this experiment 50 times and average results.

INSTRUCTIVE AND PROBLEMATIC EXAMPLE / 2



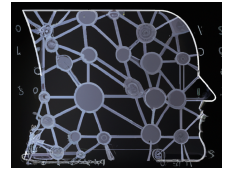
- Plotted is the best "tuning error" (i.e. the performance of the model with fixed λ as evaluated by the cross-validation) after k tuning iterations.
- We have performed the experiment for different sizes of learning data that were cross-validated.

INSTRUCTIVE AND PROBLEMATIC EXAMPLE / 3



- For 1 experiment, the CV score will be nearly 0.5, as expected
- We basically sample from a (rescaled) binomial distribution when we calculate error rates
- And multiple experiment scores are also nicely arranged around the expected mean 0.5

NESTED RESAMPLING

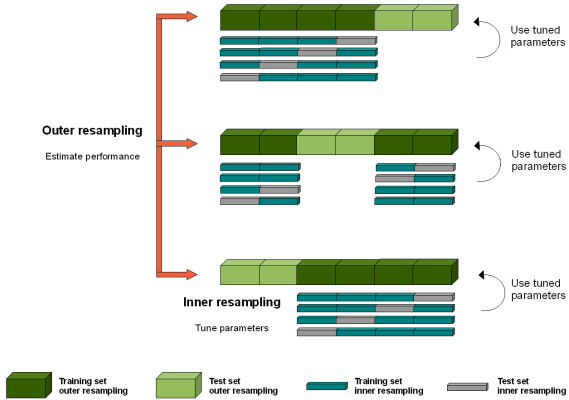


Just like we can generalize hold-out splitting to resampling to get more reliable estimates of the predictive performance, we can generalize the training/validation/test approach to **nested resampling**.

This results in two nested resampling loops, i.e., resampling strategies for both tuning and outer evaluation.

NESTED RESAMPLING / 2

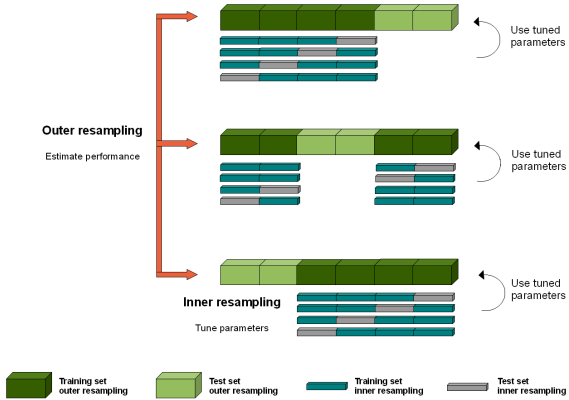
Assume we want to tune over a set of candidate HP configurations $\lambda_i; i = 1, \ldots$ with 4-fold CV in the inner resampling and 3-fold CV in the outer loop. The outer loop is visualized as the light green and dark green parts.



NESTED RESAMPLING / 3

In each iteration of the outer loop we:

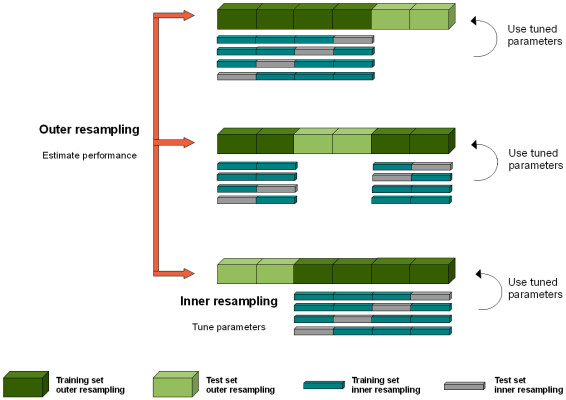
- Split off the light green testing data
- Run the tuner on the dark green part of the data, e.g., evaluate each λ_i through fourfold CV on the dark green part



NESTED RESAMPLING / 4

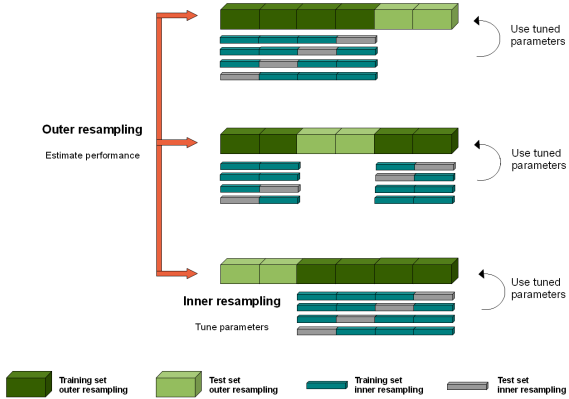
In each iteration of the outer loop we:

- Return the winning λ^* that performed best on the grey inner test sets
- Re-train the model on the full outer dark green train set
- Evaluate it on the outer light green test set



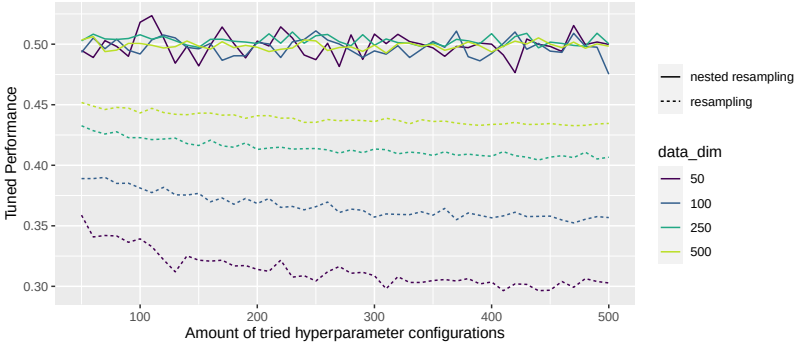
NESTED RESAMPLING / 5

The error estimates on the outer samples (light green) are unbiased because this data was strictly excluded from the model-building process of the model that was tested on.



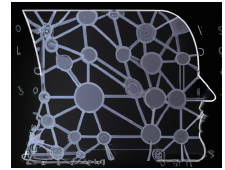
NESTED RESAMPLING - INSTRUCTIVE EXAMPLE

Taking again a look at the motivating example and adding a nested resampling outer loop, we get the expected behavior:

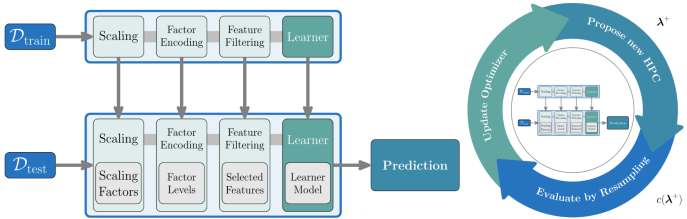


Pipelines

PIPELINES AND AUTOML



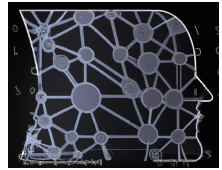
- ML typically has several data transformation steps before model fit
- If steps are in succession, data flows through sequential pipeline
- NB: Each node has a train and predict step and learns params
- And usually has HPs

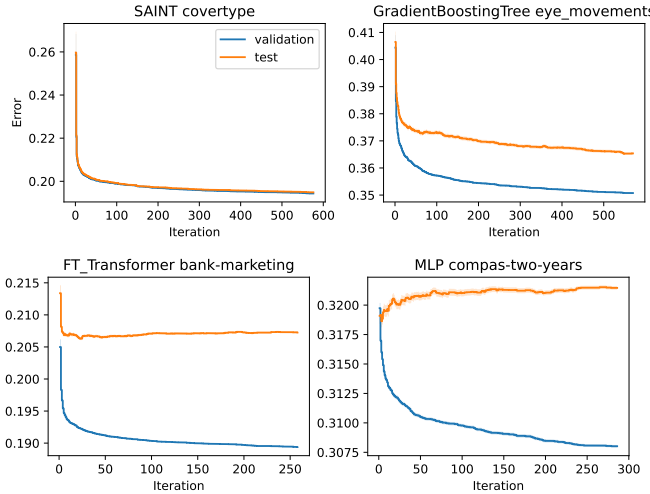


Pipelines are required to embed full model building into CV to avoid overfitting and biased evaluation!

Practical Considerations

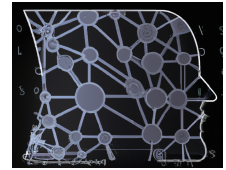
VALID-TEST CURVES IN HYPERPARAMETER OPTIMIZATION





Top Left: Idea, Top Right: Meta-Overfitting, Bottom Left: Begin Overtuning, Bottom Right: Severe Overtuning

OVERTUNING



Definition

Given a trajectory $(\boldsymbol{\lambda}_1, \ldots, \boldsymbol{\lambda}_T)$, we define for each time point $1 \leq t \leq T$:

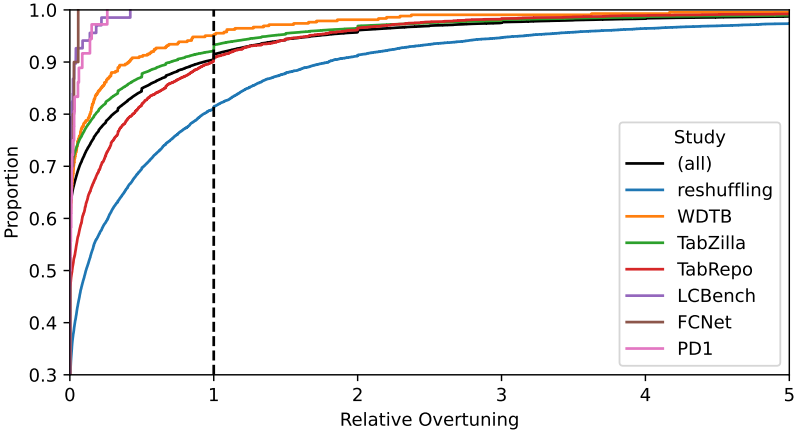
$$\textit{overtuning:} \quad \mathrm{ot}_t(\boldsymbol{\lambda}_1, \ldots, \boldsymbol{\lambda}_t, \ldots, \boldsymbol{\lambda}_T) = \mathrm{test}(\boldsymbol{\lambda}_t^*) - \min_{\boldsymbol{\lambda}_{t'}^* \in \{\boldsymbol{\lambda}_1^*, \ldots, \boldsymbol{\lambda}_t^*\}} \mathrm{test}(\boldsymbol{\lambda}_{t'}^*) \quad (1)$$

$$\textit{meta-overfitting:} \quad \mathrm{of}_t(\boldsymbol{\lambda}_1, \ldots, \boldsymbol{\lambda}_t, \ldots, \boldsymbol{\lambda}_T) = \mathrm{test}(\boldsymbol{\lambda}_t^*) - \widehat{\mathrm{val}}(\boldsymbol{\lambda}_t^*)[1\mathit{ex}] \quad (2)$$

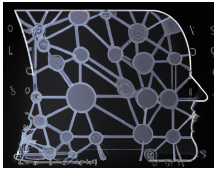
where $\widehat{\mathrm{test}}(\boldsymbol{\lambda}_t^*) := \widehat{\mathrm{GE}}(\mathcal{I}, \boldsymbol{\lambda}_t^*, \mathcal{J}_{\mathit{test}}, \rho)$ and $\widehat{\mathrm{val}}(\boldsymbol{\lambda}_t^*) := \widehat{\mathrm{GE}}(\mathcal{I}, \boldsymbol{\lambda}_t^*, \mathcal{J}, \rho)$

Question: is meta-overfitting or overtuning more problematic?

OVERTUNING


1.0
0.9
0.8
0.7
0.6
0.5
0.4
0.3
Proportion
0
1
2
3
4
5
Relative Overtuning
Study
(all)
reshuffling
WDTB
TabZilla
TabRepo
LCBench
FCNet
PD1

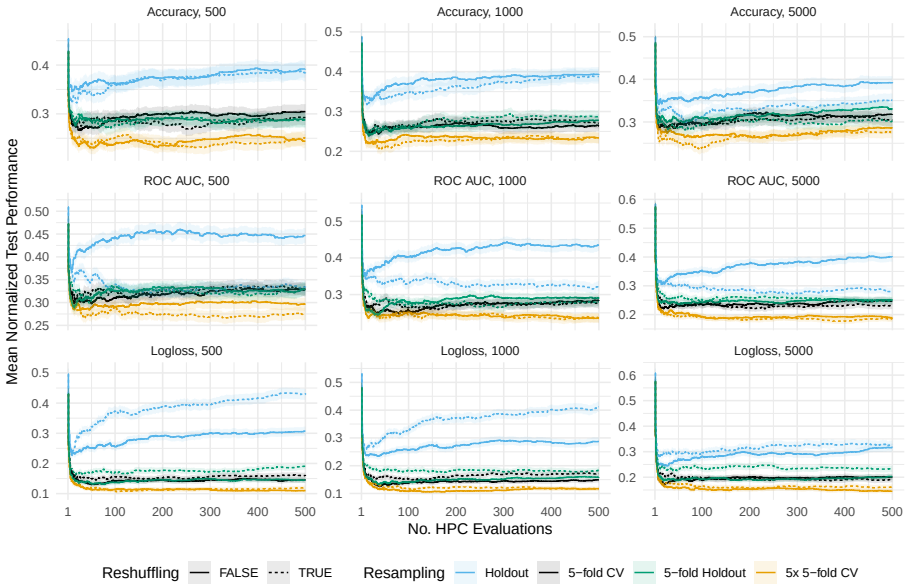
REMEDIES AGAINST OVERTUNING

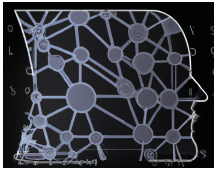


or recipes for better generalization performance

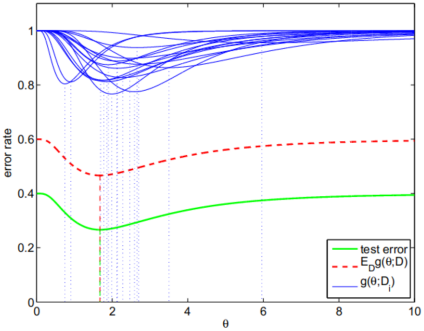
- More data
- Cross-validation, repeated cross-validation
- Bayesian optimization

VISUALIZATION

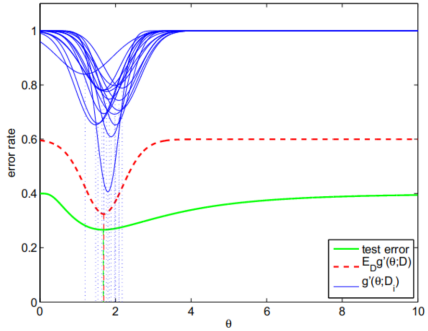




VARIANCE IN RESAMPLING



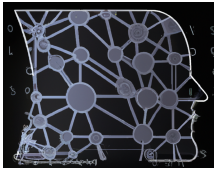
(a)



(b)

Recap

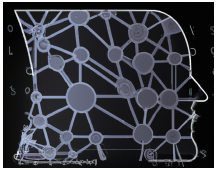
IMPORTANCE OF CV ON KAGGLE



- Understand the data - After you download the data, start exploring features. Look at data types. Check variable classes. Create some univariate - bivariate plots to understand the nature of variables.
- Understand the metric to optimise - Every problem comes with a unique evaluation metric. It's imperative for you to understand it, specially how does it change with target variable.
- Decide cross validation strategy - To avoid overfitting, make sure you've set up a cross validation strategy in early stages. **A nice CV strategy will help you get reliable score on leaderboard**.
- Start hyper parameter tuning

Source: KazAnova, former Kaggle Champion at `https://www.hackerearth.com/practice/machine-learning/advanced-techniques/winning-tips-machine-learning-competitions-kazanova-current-kaggle-3/tutorial/`

TAKE-HOME MESSAGES



- Conduct correct cross-validation
- Nested-Resampling for performance estimation
- Use advanced and robust HPO
- Include your preprocessing in the splitting